

Section Informatique

Module : Développement Mobile

Projet : Application Mobile de Gestion d'Interventions de Terrain ("FieldTask Pro")

Présenté par :

Abdoulaye Thiaw Diouf P32 677

Kouna Samb P32 300

Aby Mbaye P32 864

Superviseur :

Pr Gaye

Année Académique : 2025/ 2026

1. Présentation du projet

TechFix gère des techniciens itinérants qui travaillent souvent dans des zones à faible couverture réseau. **FieldTask Pro** est le prototype d'une application Flutter offline-first : le technicien consulte ses interventions, change leur statut, joint des photos commentées et recueille la signature du client, même sans connexion. Les modifications sont conservées localement puis synchronisées automatiquement avec le serveur dès le retour du réseau. Principales technologies utilisées :

Besoin	**Solution Retenue**
État de l'application	flutter_bloc (Cubits)
Persistance locale	SQLite via sqlite (FFI sur Windows/Linux/macOS, WebAssembly sur navigateur)
Détection du réseau	connectivity_plus
Carte et position	flutter_map + latlong2, geolocator
Photos	image_picker (caméra ou galerie), path_provider
Notifications	flutter_local_notifications + timezone
Session	flutter_secure_storage (jeton) + SQLite (utilisateur)
Backend de test	JSON-Server (fichier mock_api/db.json)

2. Architecture et arborescence du projet

Le code est séparé en trois couches (**Data**, **Domain**, **Presentation**), plus un dossier core pour les éléments transverses (couleurs, thème, services, utilitaires). Cette séparation permet de modifier l'interface, la base locale ou l'API sans réécrire le reste, et de tester chaque partie isolément.

```
├──core
│   ├──config
│   │       api_config.dart
│   │
│   ├──constants
│   │       app_colors.dart
│   │
│   ├──services
│   │       connectivity_service.dart
│   │       notification_service.dart
│   │       token_store.dart
│   │
│   ├──theme
│   │       app_theme.dart
│   │
│   └──utils
│           confirm.dart
│           toast.dart
│
├──data
│   ├──local
│   │       database_helper.dart
│   │       local_storage.dart
│   │
│   ├──remote
│   │       remote_api.dart
│   │
│   └──repositories
│           auth_repository.dart
│           interventionRepositoryImpl.dart
│           intervention_repository.dart
│
├──domain
│   ├──entities
│   │       intervention.dart
│   │       intervention_status.dart
│   │       user.dart
│   │
│   ├──repositories
│   │       auth_repository_interface.dart
│   │       intervention_repository_interface.dart
│   │
│   └──usecases
│           auth_usecases.dart
│           intervention_usecases.dart
│
└──presentation
    ├──blocs
    │       auth_cubit.dart
    │       intervention_cubit.dart
    │       sync_cubit.dart
    │
    ├──pages
    │       dashboard_page.dart
    │       intervention_detail_page.dart
    │       login_page.dart
    │       map_page.dart
    │       profile_page.dart
    │       signature_page.dart
    │       splash_page.dart
    │
    └──widgets
            network_banner.dart
            photo_section.dart
            reminders_dialog.dart
```

Rôle de chaque couche

Couche	Contenu	Justification
Domain	Entités métier, interfaces de repository et use cases, sans dépendance à l'interface.	Le cœur du métier reste stable et testable sans émulateur.
Data	Base SQLite, API REST, repository qui décide entre local et distant.	Un seul endroit connaît la stratégie offline-first ; l'interface ne parle jamais directement à l'API.
Presentation	Pages, widgets et Cubits.	L'interface réagit aux états émis par les Cubits ; elle ne contient pas de logique de synchronisation.
Core	Couleurs, thème, services de connectivité et de notification, utilitaires.	Éléments partagés par plusieurs couches, sans logique métier.

3. Gestion d'état avec BloC / Cubit

La logique globale est gérée par trois Cubits fournis par un **MultiBlocProvider** dans main.dart. **setState** est réservé à l'état purement visuel d'un écran (filtre et recherche du tableau de bord, affichage du mot de passe, tracé de la signature). Les Cubits n'appellent plus directement le stockage ou l'API : ils passent par des cas d'utilisation (**GetInterventions**, **ChangeInterventionStatus**, **LoginUseCase**, etc.) de la couche Domain, qui dépendent d'interfaces de repository implémentées dans la couche Data.

Cubit	Responsabilité	États principaux
AuthCubit	Connexion, restauration de la session au démarrage, déconnexion, modification du profil.	AuthLoading, AuthAuthenticated, AuthError
InterventionCubit	Chargement de la liste (local, puis serveur si possible), changement de statut.	InterventionLoading, InterventionLoaded(all), InterventionError(message)
SuncCubit	Écoute la connectivité et vide la file de synchronisation ; recharge la liste à la fin.	SyncOnline, SyncOffline, SyncInProgress, SyncCompleted

4. Stratégie offline-first et synchronisation

Persistance locale : Toutes les données sont stockées dans SQLite. Au démarrage, LocalStorage.init() charge les interventions et la file d'attente en mémoire ; les lectures se font depuis ce cache et chaque écriture est immédiatement enregistrée dans SQLite. L'application fonctionne donc de la même façon avec ou sans réseau.

Table	Rôle
Interventions	Missions du technicien, avec un indicateur synced.
sync_queue	File d'attente : entityType, entityId, action, payload (JSON), createdAt.
photos	Chemin local de chaque photo, texte d'explications, date. Base en version 3 : la colonne explications a été ajoutée par une migration automatique.
signatures	Signature du client (image PNG encodée en base64), date.
session	Utilisateur connecté, pour rouvrir l'application sans nouvelle connexion.

File d'attente : Chaque action faite hors ligne (changement de statut, ajout ou suppression d'une photo, modification d'explications, signature, modification du profil) est enregistrée dans sync_queue avec son action (création, modification ou suppression) et ses données. L'ordre d'insertion est conservé : les éléments sont envoyés dans l'ordre où ils ont été faits.
Déclenchement : ConnectivityService (connectivity_plus) signale les changements de réseau. SyncCubit les écoute : quand la connexion revient, il parcourt la file et envoie chaque élément au serveur par RemoteApi. Un élément est retiré de la file seulement après un envoi réussi. Si le serveur est éteint ou injoignable, l'élément reste et sera renvoyé plus tard. À la fin, la liste est rechargée depuis la base locale. Un bandeau (NetworkBanner) indique en permanence si l'application est en ligne ou hors ligne.

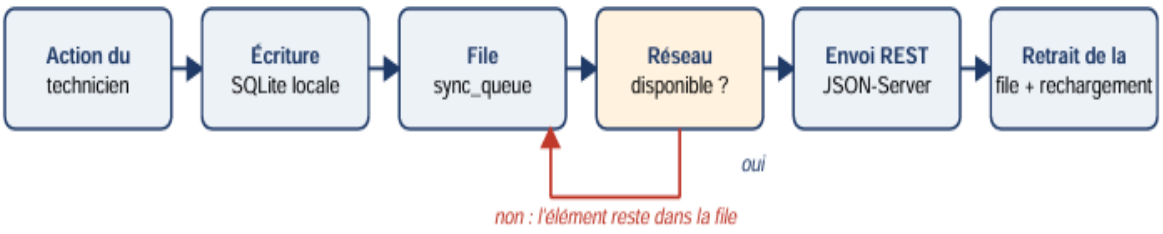


Figure 1 - Cycle d'une modification, de l'écriture locale à la synchronisation.

Conflits : Il n'y a pas de résolution de conflits avancée : le serveur de test conserve la dernière modification reçue. C'est suffisant pour un technicien qui travaille sur ses propres missions.
Photos et signatures : JSON-Server n'accepte que du JSON ; la signature est donc envoyée sous forme de texte base64. Les photos sont enregistrées localement (chemin du fichier) et déclarées au serveur avec leur explication.

5. Fonctionnalités et interactions natives

Authentification et profil : L'écran de connexion vérifie l'email et le mot de passe auprès de la collection users de JSON-Server. Le serveur renvoie un jeton JWT simulé, conservé avec flutter_secure_storage : la session n'est restaurée au démarrage que si ce jeton existe. Les données de l'utilisateur sont gardées dans la table session de SQLite. L'en-tête du tableau de bord affiche le nom de l'utilisateur connecté et ses initiales. L'écran Profil permet de modifier nom, email et téléphone ; la modification passe par la file de synchronisation (PATCH sur l'utilisateur).
Tableau de bord : Liste des interventions avec filtres par statut (Toutes, En attente, En cours, Terminées), recherche textuelle dynamique (identifiant, client, adresse) et bandeau d'état du réseau.
Fiche d'intervention et règle de clôture : La fiche affiche client, adresse, description, équipement et rendez-vous. Le statut évolue de En attente à En cours, puis Terminée. Pour terminer, il faut au moins une photo et la signature du client : une liste de contrôle indique ce qui manque et le bouton reste grisé tant que ce n'est pas fait. Une fois l'intervention terminée, photos, explications et signature sont verrouillées en lecture seule, car elles servent de preuve.
Photos commentées : Le bouton d'ajout propose l'appareil photo ou la galerie (image_picker). Chaque photo reçoit un champ Explications (300 caractères maximum, modifiable tant que l'intervention n'est pas terminée). Une photo peut être supprimée après confirmation ; la suppression est aussi envoyée au serveur.
Signature : Le client signe avec le doigt sur un canevas (CustomPaint). Le tracé est converti en image PNG sur fond blanc, enregistré en base64 dans SQLite, puis envoyé par la file de synchronisation. La fiche affiche la signature avec la mention Signé ; elle peut être effacée et refaite tant que l'intervention n'est pas terminée.
Carte : La page Carte affiche les interventions sur une carte flutter_map à partir de leurs coordonnées, et la position courante du technicien est obtenue avec geolocator (délai maximum de 10 secondes). Un appui sur un marqueur ouvre une fiche rapide de l'intervention.
Notifications locales : Un rappel est programmé 30 minutes avant l'heure de rendez-vous de chaque intervention non terminée. Les rappels sont recalculés à chaque changement de la liste : une mission terminée n'est plus rappelée, et une heure déjà passée est ignorée. Depuis la cloche du tableau de bord, une fenêtre liste les rappels prévus et propose deux boutons de test : une notification immédiate et un rappel dans une minute.

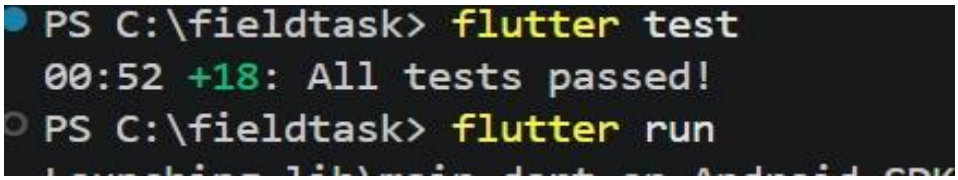
6. Backend de test

Le serveur est un **JSON-Server local** alimenté par **mock_api/db.json**, qui contient les collections users, interventions, photos et signatures. Les routes REST utilisées sont décrites ci-dessous.

Usage	**Requête**
Connexion	GET /users
Liste des interventions	GET /interventions
Changement de statut	PATCH /interventions/{id}
Photos	POST /photos, PATCH /photos/{id}, DELETE /photos/{id}
Signatures	POST /signatures, DELETE /signatures/{id}
Profil	PATCH /users/{id}

7. Tests automatisés

Les tests se lancent depuis la racine du projet avec la commande **flutter test**. Les tests sont répartis dans trois fichiers : **domain_test.dart** (entités et statuts), **usecases_test.dart** (use cases, avec des repositories factices) et **widget_test.dart** (écran de connexion, utilisateur, rappels de notification).



8. Limites connues et pistes d'amélioration

Plateformes : la caméra, les notifications et la géolocalisation précise demandent un téléphone ou un émulateur Android. Sur Windows et navigateur, les photos passent par la galerie, les notifications ne sont pas supportées, et les photos du navigateur disparaissent au rechargement. **Images sur le serveur** : envoyer la signature en base64 alourdit le JSON ; un vrai backend utiliserait un stockage de fichiers (par exemple Firebase Storage). **Couches** : les écrans de photos et de signature appellent encore directement le repository ; ils pourraient eux aussi passer par des use cases.- Conflits : ajouter un horodatage de version pour gérer deux modifications concurrentes du même enregistrement.
Jeton : le jeton JWT est simulé par le serveur de test ; un vrai serveur d'authentification le signerait et le ferait expirer.

9. Conclusion

FieldTask Pro répond au besoin principal de TechFix : un technicien peut travailler sans réseau, fournir une preuve d'intervention (photos commentées et signature du client) et retrouver ses données synchronisées dès que la connexion revient. L'architecture en trois couches et les Cubits rendent le code lisible et facile à faire évoluer.